

PRACTICE SESSION

21.09.2025

Implementation of Principal Component Analysis in Python

- **Step 1: Importing Required Libraries**

- We import the necessary library like [pandas](#), [numpy](#), [scikit learn](#), [seaborn](#) and [matplotlib](#).

- import numpy as np
- import pandas as pd
- from sklearn.preprocessing import StandardScaler
- from sklearn.decomposition import PCA
- from sklearn.model_selection import train_test_split
- from sklearn.linear_model import LogisticRegression
- from sklearn.metrics import confusion_matrix
- import matplotlib.pyplot as plt
- import seaborn as sns

• Step 2: Creating Sample Dataset

- We make a small dataset with features Height, Weight, Age and Gender.

- data = {
 - 'Height': [170, 165, 180, 175, 160, 172, 168, 177, 162, 158],
 - 'Weight': [65, 59, 75, 68, 55, 70, 62, 74, 58, 54],
 - 'Age': [30, 25, 35, 28, 22, 32, 27, 33, 24, 21],
 - 'Gender': [1, 0, 1, 1, 0, 1, 0, 1, 0, 0] # 1 = Male, 0 = Female
- }
- df = pd.DataFrame(data)
- print(df)

Output:

	Height	Weight	Age	Gender
0	170	65	30	1
1	165	59	25	0
2	180	75	35	1
3	175	68	28	1
4	160	55	22	0
5	172	70	32	1
6	168	62	27	0
7	177	74	33	1
8	162	58	24	0
9	158	54	21	0

Dataset

- **Step 3: Standardizing the Data**

- Since the features have different scales Height vs Age we standardize the data. This makes all features have mean = 0 and standard deviation = 1 so that no feature dominates just because of its units.

- `X = df.drop('Gender', axis=1)`
- `y = df['Gender']`
- `scaler = StandardScaler()`
- `X_scaled = scaler.fit_transform(df)`

- **Step 4: Applying PCA algorithm**

- Reduce the data from 3 features to 2 new features called principal components. These components capture most of the original information but in fewer dimensions.
- Split the data into 70% training and 30% testing sets.
- Train a logistic regression model on the reduced training data and predict gender labels on the test set.

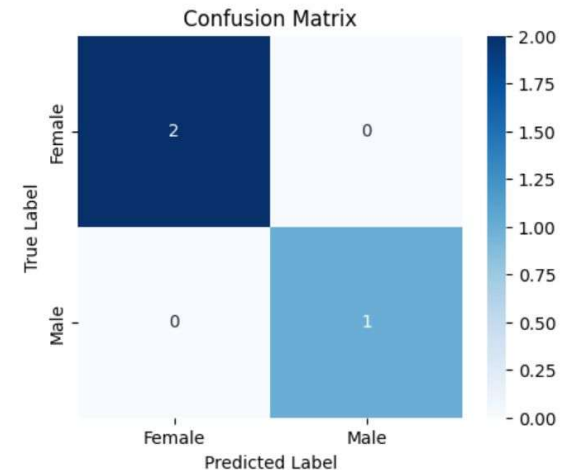
- `pca = PCA(n_components=2)`
- `X_pca = pca.fit_transform(X_scaled)`
- `X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)`
- `model = LogisticRegression()`
- `model.fit(X_train, y_train)`
- `y_pred = model.predict(X_test)`

- **Step 5: Evaluating with Confusion Matrix**

- The [confusion matrix](#) compares actual vs predicted labels. This makes it easy to see where predictions were correct or wrong.

- `cm = confusion_matrix(y_test, y_pred)`
- `plt.figure(figsize=(5,4))`
- `sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Female', 'Male'], yticklabels=['Female', 'Male'])`
- `plt.xlabel('Predicted Label')`
- `plt.ylabel('True Label')`
- `plt.title('Confusion Matrix')`
- `plt.show()`

Output:

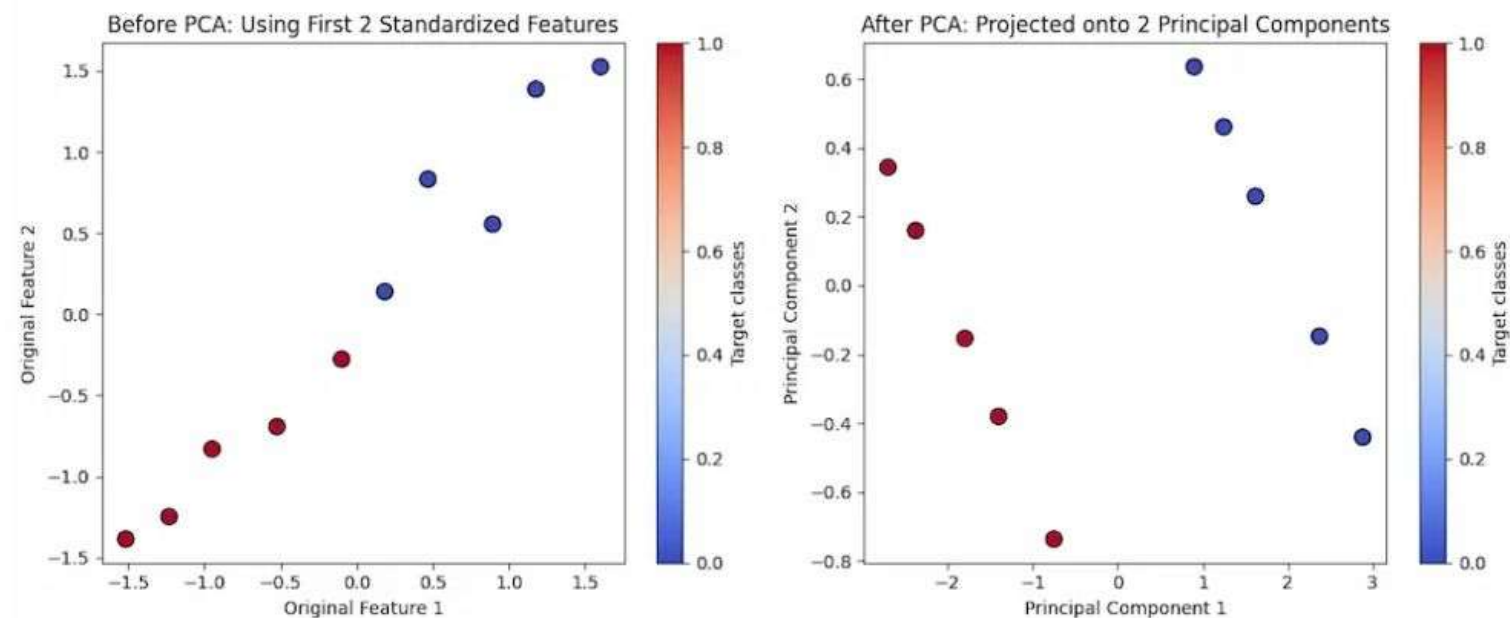


- **Step 6: Visualizing PCA Result**

- `y_numeric = pd.factorize(y)[0]`
- `plt.figure(figsize=(12, 5))`
- `plt.subplot(1, 2, 1)`
- `plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_numeric, cmap='coolwarm',
edgecolor='k', s=80)`
- `plt.xlabel('Original Feature 1')`
- `plt.ylabel('Original Feature 2')`
- `plt.title('Before PCA: Using First 2 Standardized Features')`
- `plt.colorbar(label='Target classes')`
- `plt.subplot(1, 2, 2)`
- `plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_numeric, cmap='coolwarm',
edgecolor='k', s=80)`
- `plt.xlabel('Principal Component 1')`
- `plt.ylabel('Principal Component 2')`
- `plt.title('After PCA: Projected onto 2 Principal Components')`
- `plt.colorbar(label='Target classes')`
- `plt.tight_layout()`
- `plt.show()`

- **Before PCA:** Shows the **original standardized data** plotted using the first two features. There is **no guarantee of clear separation** between classes as these are raw input dimensions.
- **After PCA:** Displays the **transformed data** using the **top 2 principal components**. These new components capture the **maximum variance** often showing better **class separation and structure** making it easier to analyze or model

Output:



Implementation of LDA using Python

- Perform linear discriminant analysis using Scikit-learn library on the Iris dataset.
- **1. Importing Required Libraries**
- Import all necessary libraries for data processing, visualization, dimensionality reduction, and modeling
- [NumPy](#): for numerical operations and array manipulations
- [Pandas](#): for creating and managing structured datasets
- [Matplotlib.pyplot](#): for plotting and visualizations
- [Scikit-learn](#): for machine learning tools like dataset loading, preprocessing, dimensionality reduction, and classifiers

- import numpy as np
- import pandas as pd
- import matplotlib.pyplot as plt
- from mpl_toolkits.mplot3d import Axes3D
- from sklearn.datasets import load_iris
- from sklearn.preprocessing import StandardScaler, LabelEncoder
- from sklearn.model_selection import train_test_split
- from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
- from sklearn.ensemble import RandomForestClassifier
- from matplotlib.colors import ListedColormap

- **2. Loading the Dataset**

- We load the Iris dataset and convert it into a Pandas DataFrame. Features X and target labels y are separated
- **pd.DataFrame(columns=..., data=...)** creates a DataFrame from raw data
- **iloc[:, 0:4]** selects first four columns as features
- **iloc[:, 4]** selects the target column

- `iris = load_iris()`
- `dataset = pd.DataFrame(columns=iris.feature_names, data=iris.data)`
- `dataset['target'] = iris.target`

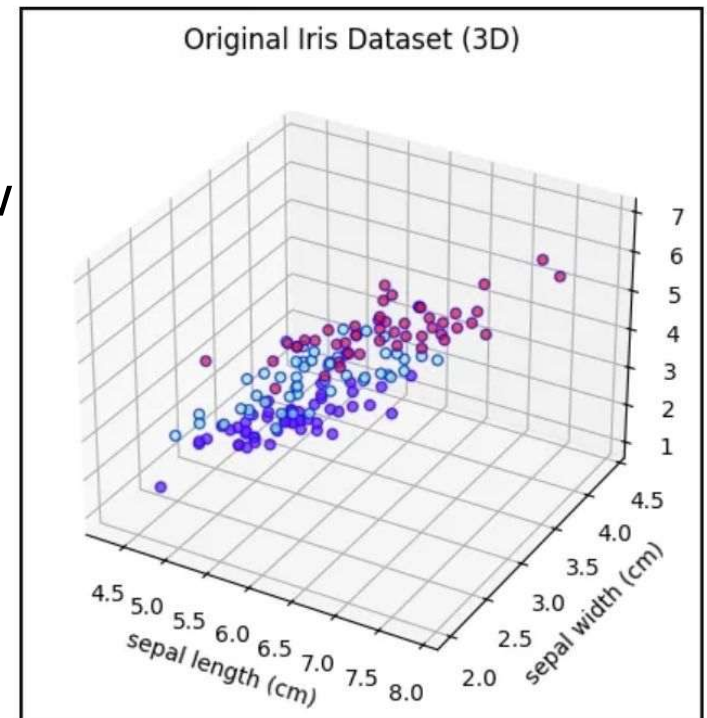
- `X = dataset.iloc[:, 0:4].values`
- `y = dataset.iloc[:, 4].values`

3. Data Preprocessing

- Scale the features and encode the target labels, then split the dataset into training and testing sets
 - **StandardScaler().fit_transform(X)** scales features
 - **LabelEncoder().fit_transform(y)** encodes categorical labels as integers
 - **train_test_split(X, y, test_size=0.2, random_state=42)** splits data into 80% train and 20% test.
-
- `sc = StandardScaler()`
 - `X_scaled = sc.fit_transform(X)`
-
- `le = LabelEncoder()`
 - `y_encoded = le.fit_transform(y)`
-
- `X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42)`

4. Visualizing the Original Iris Dataset in 3D

- Create a 3D scatter plot using the first three features to visualize the original data distribution
- Axes3D enables 3D plotting
- **scatter(..., c=y, cmap='rainbow', alpha=0.7, edgecolors='b')** colors points by class
- `fig = plt.figure(figsize=(7,5))`
- `ax = fig.add_subplot(111, projection='3d')`
- `ax.scatter(X[:,0], X[:,1], X[:,2], c=y, cmap='rainbow', alpha=0.7, edgecolors='b')`
- `ax.set_xlabel(iris.feature_names[0])`
- `ax.set_ylabel(iris.feature_names[1])`
- `ax.set_zlabel(iris.feature_names[2])`
- `ax.set_title('Original Iris Dataset (3D)')`
- `plt.show()`



5. Random Forest Decision Boundary Without LDA

- Train a [Random Forest](#) on the original features (first two features only for visualization) and plot the decision boundary
- **RandomForestClassifier(max_depth=2, random_state=0)** initialize classifier
- **.fit(X_train_2D, y_train)** train classifier
- **.predict()** predict over grid for decision boundary
- **contourf()** fills decision regions with colors

```
X_train_2D = X_train[:, :2]
```

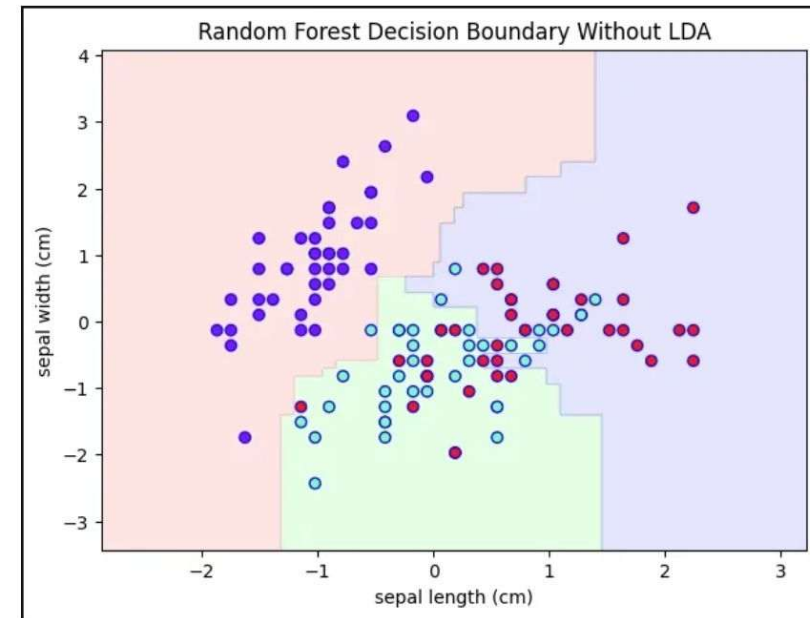
```
rf_without_lda = RandomForestClassifier(max_depth=2, random_state=0)  
rf_without_lda.fit(X_train_2D, y_train)
```

```
x_min, x_max = X_train_2D[:,0].min() - 1, X_train_2D[:,0].max() + 1  
y_min, y_max = X_train_2D[:,1].min() - 1, X_train_2D[:,1].max() + 1  
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),  
                     np.arange(y_min, y_max, 0.02))
```

```
Z = rf_without_lda.predict(np.c_[xx.ravel(), yy.ravel()])  
Z = Z.reshape(xx.shape)  
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
```

```
plt.figure(figsize=(7,5))  
plt.contourf(xx, yy, Z, alpha=0.3, cmap=cmap_light)  
plt.scatter(X_train_2D[:,0], X_train_2D[:,1], c=y_train, cmap='rainbow', edgecolors='b')  
plt.xlabel(iris.feature_names[0])  
plt.ylabel(iris.feature_names[1])  
plt.title('Random Forest Decision Boundary Without LDA')  
plt.show()
```

Output:



6. Applying LDA

- Reduce the feature space to 2 components to maximize class separability
 - **LinearDiscriminantAnalysis(n_components=2)** reduces features to 2 components
 - **.fit_transform(X_train, y_train)** fit LDA on training data and transform it
 - **.transform(X_test)** project test data onto same LDA components
-
- `lda = LinearDiscriminantAnalysis(n_components=2)`
 - `X_train_lda = lda.fit_transform(X_train, y_train)`
 - `X_test_lda = lda.transform(X_test)`

7. Random Forest Decision Boundary With LDA

- Train a Random Forest on LDA-transformed features and plot its decision boundary
- **RandomForestClassifier(max_depth=2, random_state=0)** initialize classifier
- **.fit(X_train_lda, y_train)** train classifier
- **.predict()** predict over grid for decision boundary
- **contourf()** fills decision regions with colors

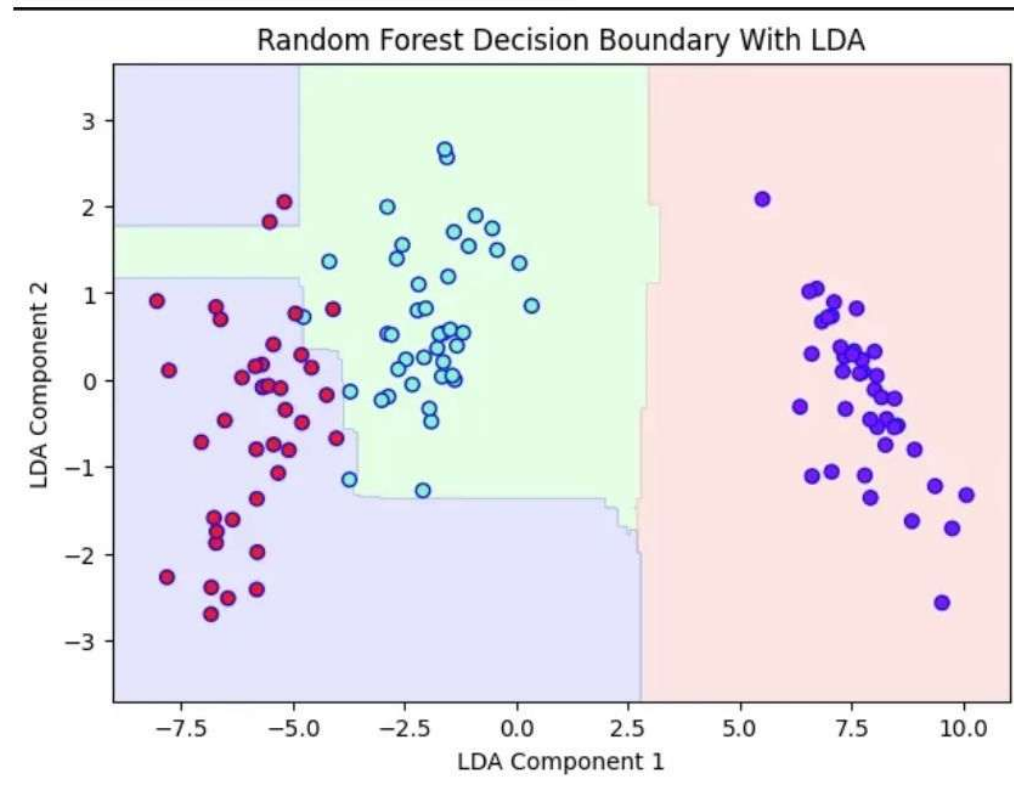
```
rf_with_lda = RandomForestClassifier(max_depth=2,  
random_state=0)  
rf_with_lda.fit(X_train_lda, y_train)
```

Output:

```
x_min, x_max = X_train_lda[:,0].min() - 1,  
X_train_lda[:,0].max() + 1  
y_min, y_max = X_train_lda[:,1].min() - 1,  
X_train_lda[:,1].max() + 1  
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),  
np.arange(y_min, y_max, 0.02))
```

```
Z = rf_with_lda.predict(np.c_[xx.ravel(), yy.ravel()])  
Z = Z.reshape(xx.shape)
```

```
plt.figure(figsize=(7,5))  
plt.contourf(xx, yy, Z, alpha=0.3, cmap=cmap_light)  
plt.scatter(X_train_lda[:,0], X_train_lda[:,1], c=y_train,  
cmap='rainbow', edgecolors='b')  
plt.xlabel('LDA Component 1')  
plt.ylabel('LDA Component 2')  
plt.title('Random Forest Decision Boundary With LDA')  
plt.show()
```



- **Advantages of LDA**

- Simple and computationally efficient.
- Works well even when the number of features is much larger than the number of training samples.
- Can handle multicollinearity.

- **Disadvantages of LDA**

- Assumes Gaussian distribution of data which may not always be the case.
- Assumes equal covariance matrices for different classes which may not hold in all datasets.
- Assumes linear separability which is not always true.
- May not always perform well in high-dimensional feature spaces.